جامعة بيرزيت

BIRZEIT UNIVERSITY

Faculty of Engineering and Technology
Computer Science Department

COMP438 - Distributed Computing

Project Report

Peer-to-Peer Chat Application

Prepared by:

Students Name:	**IDs:**
Ayah Abdalmuti	1212074
Somaya DarMousa	1210323

Instructor: Dr. Mohammad Helal.

Date: 23. May. 2025

Abstract

This project implements a Peer-to-Peer (P2P) chat application using Java. A central Directory Server assists clients in discovering each other by exchanging IP addresses and port numbers. Upon activation, each client establishes a TCP connection with the Directory Server and periodically sends an "I am alive" message, allowing the server to maintain an updated list of active users. Clients receive this list and can request to connect to any peer, after which the server provides the necessary connection details. The application uses multi-threading to enable real-time, asynchronous message exchange. This project demonstrates key networking concepts such as socket programming, concurrency, and decentralized peer communication.

Table of Contents

Introduction 4
1. Project Design 4
2. Number of Sockets and Concurrency Management 6
3. How the Client Establishes Connection with Other Clients 8
4. Application Layer Header 11
Conclusion 12

Introduction

Peer-to-Peer (P2P) communication has become a popular approach for direct data exchange between users without relying heavily on central servers. This project implements a P2P chat system in Java that uses a Directory Server to help clients find and connect to each other by sharing IP addresses and ports. Each client regularly informs the server that it is active by sending an “I am alive” message. The server keeps an updated list of active clients and shares this list with all connected users. Once a user selects a peer to chat with, the application establishes a direct TCP connection for real-time messaging. The system uses multithreading to handle multiple message exchanges simultaneously, ensuring smooth and asynchronous communication between peers.

1. Project Design

The project consists of two main parts:

- **Directory Server:**

This server keeps track of active clients on the network. It receives periodic “I am alive” messages from each client and stores their IP address and port number. The server uses multithreading to handle multiple clients at the same time, ensuring the active client list is updated accurately and on time. It periodically sends the list of active client names to all connected clients. When a client requests to connect to another client, the server sends the IP address and port number of the requested client.

- **Client Process:**

  Each client connects to the Directory Server when it starts and sends an "I am alive" message every second to stay registered as active. The client receives regular updates of the active clients list from the server. The user can select another client from the list to start a chat. When a client is chosen, the client requests the IP and port of the selected client from the Directory Server. After receiving this information, the client creates a direct TCP connection with the other client to start the chat session. The client application uses multithreading to handle communication smoothly by separating the sending and receiving of messages into different threads, allowing real-time communication without delay or interruption.

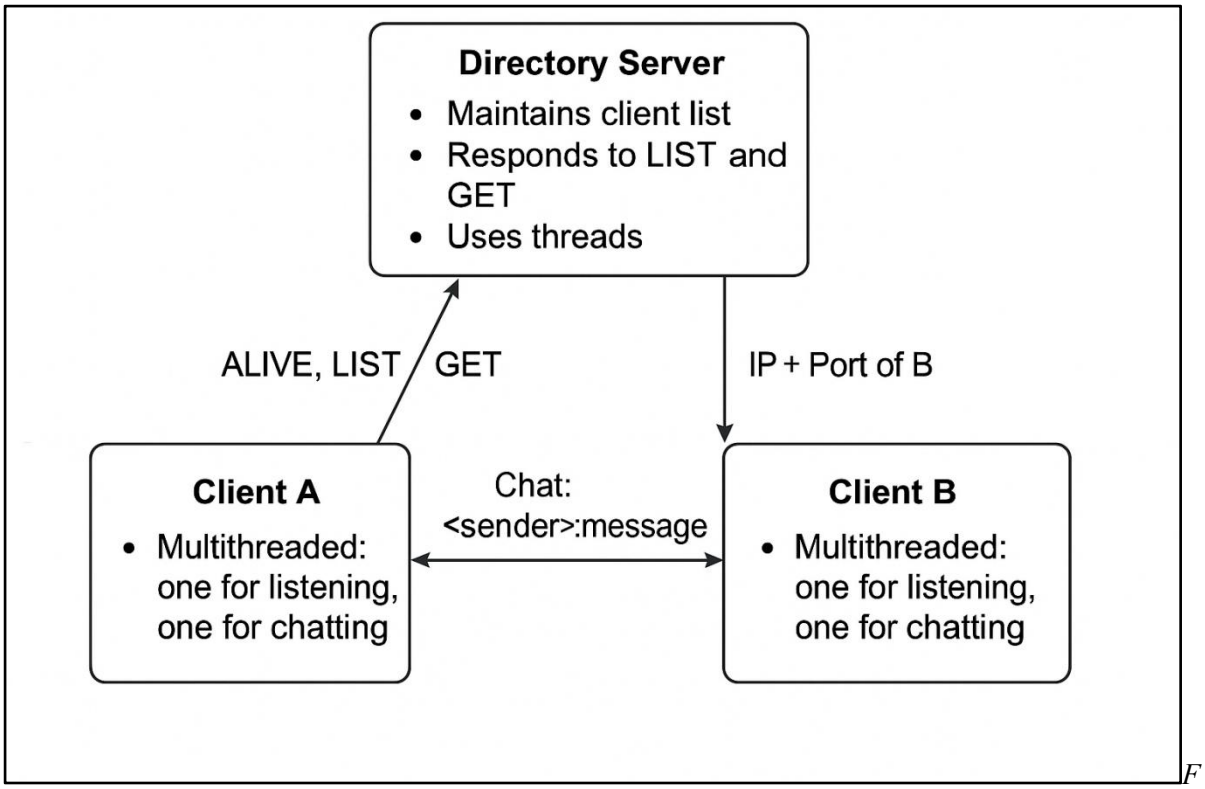

Figure 1: System Architecture showing Directory Server and Peer-to-Peer Chat.

2. Number of Sockets and Concurrency Management

This project uses multiple sockets to allow communication between the Directory Server and the clients, as well as between the clients themselves:

➢ **On the Directory Server side:**

- A single ServerSocket is used to listen for incoming client connections.
- For every client that connects, a separate Socket is created to handle communication.
- Each client connection is handled on its own thread to make sure the server can serve multiple clients at the same time.

➢ **On the Client side:**

- A socket is opened to connect with the Directory Server for sending "I am alive" messages and receiving the list of active users.
- Another ServerSocket is opened to listen for incoming chat requests from other clients.
- When the user chooses to chat with another client, a new Socket is created to establish a direct peer-to-peer TCP connection.

➢ **Concurrency Management:**

Multithreading is used to ensure smooth and real-time communication. The client application runs multiple threads:

- One thread to handle incoming and outgoing messages during a chat.
- One thread to receive updates from the Directory Server.
- One thread to periodically send the "I am alive" heartbeat to the server.

This approach allows the application to handle all tasks at the same time without delays or blocking the user interface.

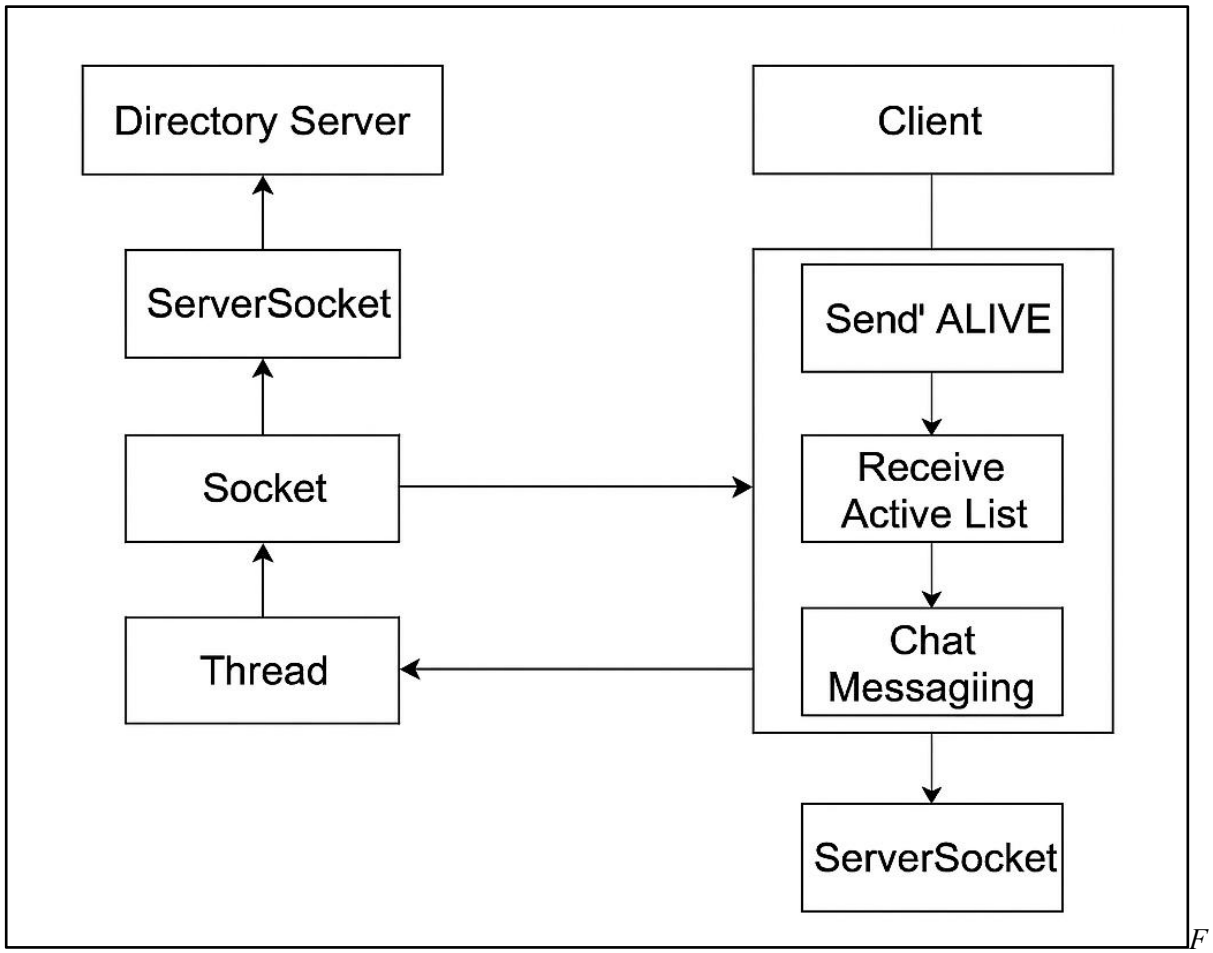

Figure 2: Socket usage and concurrency management in Directory Server and Clients.

3. How the Client Establishes Connection with Other Clients

The process of initiating a chat connection between clients involves several coordinated steps, relying on both the Directory Server and direct peer-to-peer communication:

Step-by-step Connection Process:

1. **Client Registration:**
    - When a client starts, it connects to the Directory Server and sends a heartbeat message every second to stay marked as active.
2. **Receiving Active Clients List:**
    - The Directory Server periodically sends an updated list of active clients (by name) to all connected clients.
3. **User Selection:**
    - The user selects a peer from the list to start a chat.
4. **Requesting Peer Info:**
    - The client sends a request to the Directory Server asking for the IP address and port number of the selected client.
5. **Establishing Direct TCP Connection:**
    - After receiving the target client's network information, the client opens a new TCP socket connection directly to the peer's IP and port.

6. **Chat Session Begins:**

    - The chat session is handled by separate threads for sending and receiving messages to allow smooth, asynchronous conversation.

Important Notes:

- This method ensures that the Directory Server is only used for discovery, and the actual chat happens directly between peers (true P2P).

- Multiple chats can be handled at the same time, thanks to the use of multithreading.

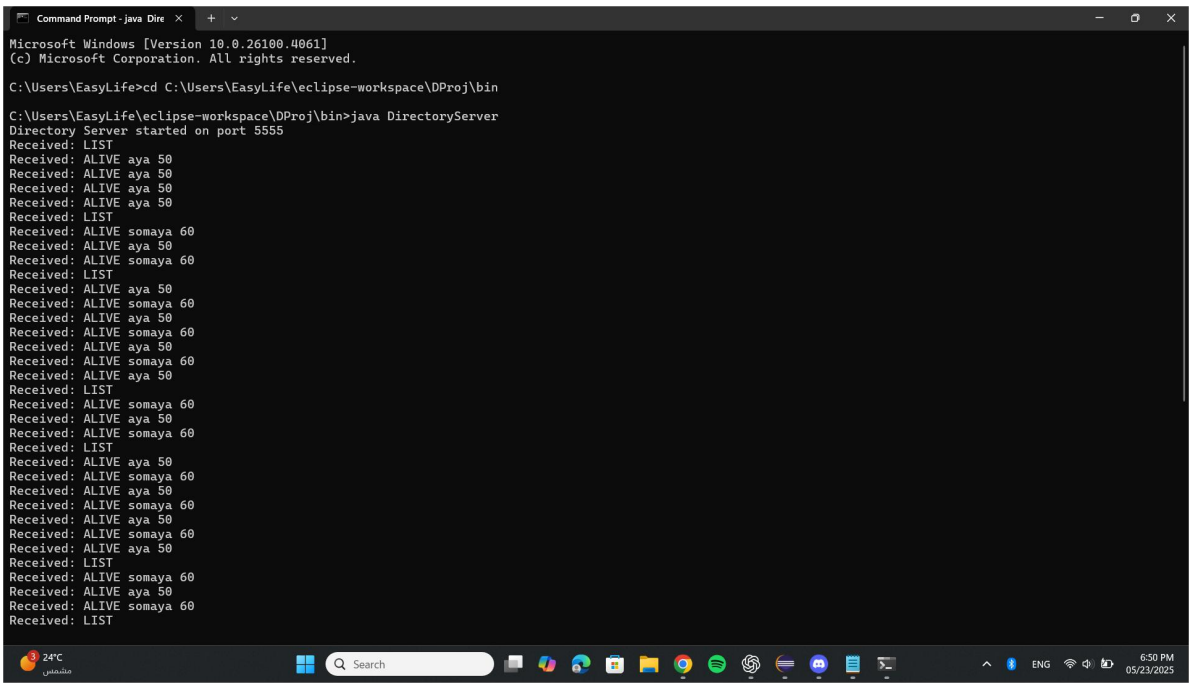

Figure 3: Directory Server Receiving Client Alive Messages.

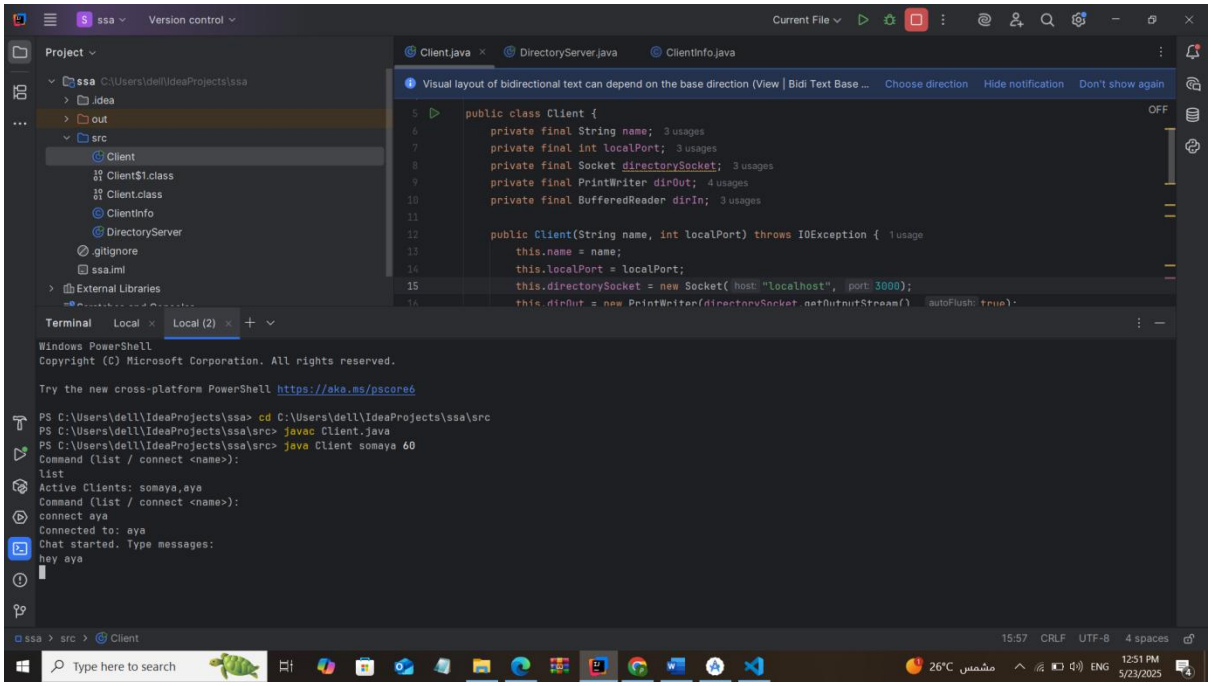

Figure 4: Client Soamay sending messages to Client Aya.

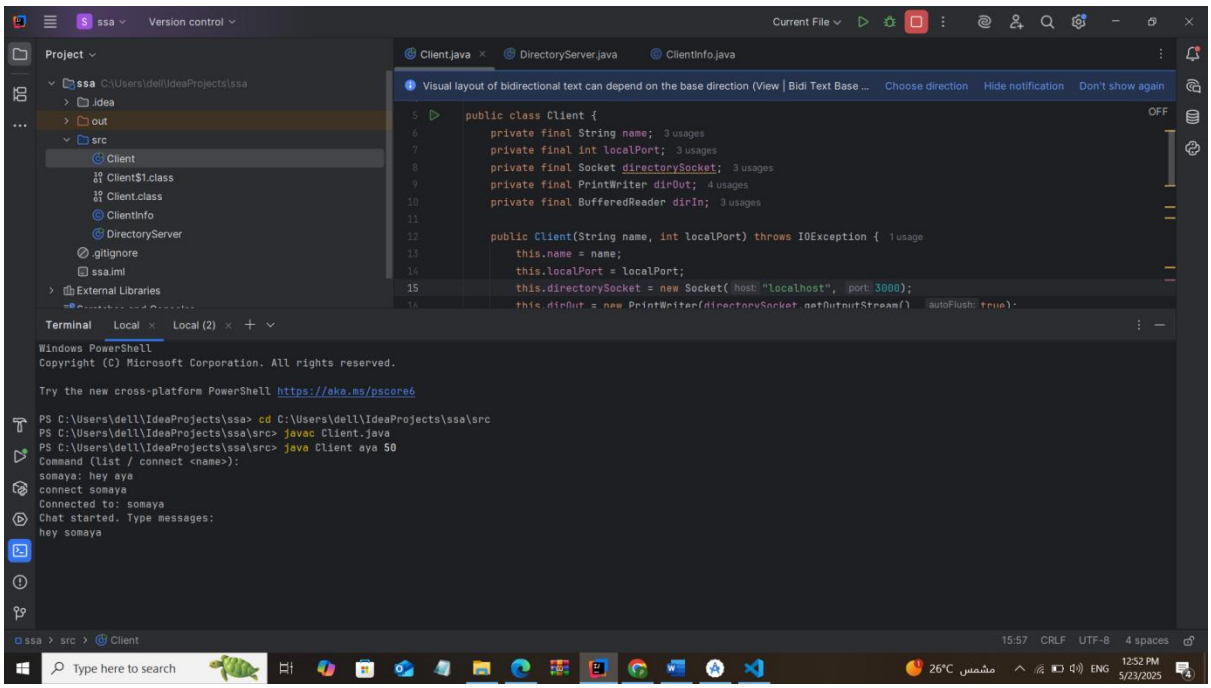

Figure 5: Client Aya receiving and replying to messages from Client Somaya.

4. Application Layer Header

- **Between Client and Directory Server:**
  - ALIVE <name> <port> – Sent every second by clients to register or update their status.
  - LIST – Clients request the current list of active users.
  - GET <name> – Clients request the IP address and port of a specific client.
  - The server responds with either <IP> <port> or NOT_FOUND.
- **Between Clients (Peer-to-Peer Chat):**
  - Messages are sent in this format: <sender>: <message>
  - This helps the receiving client know who sent the message.

This header format is simple but effective. It avoids complexity while supporting the core functionalities of the chat system, including client discovery, connection setup, and message exchange.

```java
    private void startHeartbeat() {
        new Timer(true).scheduleAtFixedRate(new TimerTask() {
            public void run() {
                dirOut.println("ALIVE " + name + " " + localPort);
            }
        }, 0, 1000);
    }
```

Figure 6: Sending ALIVE from client.

```java
        case "ALIVE": {
            String name = parts[1];
            int port = Integer.parseInt(parts[2]);
            clients.put(name, new ClientInfo(name, socket.getInetAddress().getHostAddress(), port));
            break;
        }
```

Figure 7: Directory Server processing ALIVE messages

Conclusion

This project successfully implements a peer-to-peer chat system supported by a Directory Server for client discovery. It uses TCP sockets and multithreading to enable efficient, real-time communication between clients. The designed application layer protocol is simple yet effective, ensuring smooth message exchange and connection setup. This project clearly demonstrates important networking concepts such as socket programming, concurrency management, and protocol design.